

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

—o0o—



BÁO CÁO BÀI TẬP LỚN

Môn: Kiến trúc máy tính

Chủ đề: Thiết kế Balloon Shooting Game bằng Assembly

Giảng viên: Trần Tiến Công

Nguyễn Trường Thái	B23DCCN747
Hà Duy Khánh	B23DCCN426
Vũ Quang Huy	B23DCCN412
Lê Huy Hải	B23DCCN272

Hà Nội, 5/2025

Mục lục

Danh sách hình vẽ	ii
Bảng phân công công việc chi tiết	iii
1 Giới Thiệu Tổng Quan Về Trò Chơi	1
2 Hướng Dẫn Cách Chơi Và Điều Khiển	2
3 Kết Quả Thực Thi Và Hình Ảnh Minh Họa Trò Chơi	4
4 Phân Tích Cấu Trúc Mã Nguồn Balloon Shooting Game	7
5 Phân Tích Chi Tiết Chức Năng	10
5.1 Chi tiết các Thủ tục Chính (Procedures)	10
5.2 Giải thích các Biến (Variables) Dữ liệu Chính	10
5.3 Phân tích các Nhãn (Labels) Quan trọng	11
6 Tổng Kết, Đánh Giá Và Hướng Phát Triển	20
A Phụ lục mã nguồn	22
Tài liệu tham khảo	31

Danh sách hình vẽ

3.1	Giao diện menu chính của trò chơi.	4
3.2	Giao diện trong trò chơi.	5
3.3	Toàn cảnh trò chơi đang diễn ra trong trình giả lập Emu8086, bao gồm màn hình game, mã nguồn và các thanh ghi.	5
3.4	Màn hình thông báo chiến thắng khi người chơi đạt đủ điểm. . . .	6
3.5	Màn hình thông báo thua cuộc khi người chơi để lọt quá nhiều bóng bay.	6

BẢNG PHÂN CÔNG CÔNG VIỆC CHI TIẾT

Thứ tự	Nhiệm vụ	Phạm vi	Mục tiêu
Khánh	Khởi tạo, Menu và Luồng Điều Khiển Chính Ban Đầu	• <code>main proc</code>: Thiết lập ban đầu, khởi tạo biến, gọi hàm chính. • <code>game_menu proc</code>: Logic menu (hiển thị, chờ Enter), gọi <code>clear_screen</code>, <code>show_score</code>, chuyển tiếp sang <code>main_loop</code>.	Hiểu cách khởi động, menu hoạt động, chuyển vào vòng lặp chính.
Huy	Vòng Lặp Chính và Xử Lý Đầu Vào Người Chơi	• <code>main_loop proc</code>: Cấu trúc vòng lặp chính, điều phối hàm con. • <code>key_pressed proc</code>: Xử lý phím W, S, Space, Left. • <code>inside_loop proc</code>: Cập nhật vị trí người chơi.	Hiểu cách xử lý phím và cập nhật trạng thái người chơi.
Thái	Logic Cốt Lõi Của Game, viết báo cáo LaTeX và biên dịch báo cáo nhóm	• <code>inside_loop proc</code>: Kiểm tra thua ($\text{miss} \geq 5$), va chạm ($\text{arrow_pos} == \text{loon_pos}$), cập nhật và vẽ mũi tên/bóng bay.	Hiểu cơ chế game-play: va chạm, di chuyển, điều kiện thắng/thua.
Hải	Kết Thúc Game và Các Hàm Tiện Ích	• <code>game_over proc</code>, <code>win_game proc</code>: Thông báo, hỏi chơi lại. • <code>show_score proc</code>: Cập nhật chuỗi điểm. • <code>clear_screen proc</code>: Xóa màn hình.	Hiểu xử lý kết thúc game và các hàm tiện ích.

Chương 1

Giới Thiệu Tổng Quan Về Trò Chơi

Đề tài "Game Bắn Bóng Bay (Balloon Shooting Game) bằng Assembly" được thực hiện với mục tiêu tìm hiểu và ứng dụng ngôn ngữ lập trình Assembly để xây dựng một trò chơi đơn giản. Trò chơi mô phỏng việc người chơi điều khiển một đối tượng để bắn các bóng bay di chuyển trên màn hình. Việc lựa chọn đề tài này xuất phát từ mong muốn được thực hành các kiến thức về lập trình cấp thấp, quản lý bộ nhớ, và tương tác phần cứng cơ bản thông qua việc phát triển một ứng dụng giải trí nhỏ.

Trò chơi được phát triển dựa trên một phiên bản mã nguồn mở và đã được nhóm chỉnh sửa, cải tiến để phù hợp với yêu cầu của bài tập lớn. Qua đó, nhóm đã có cơ hội làm quen với cấu trúc của một chương trình Assembly, cách xử lý đồ họa ký tự, và logic điều khiển trong game.

Chương 2

Hướng Dẫn Cách Chơi Và Điều Khiển

Trò chơi "Balloon Shooting Game" có cách điều khiển và luật chơi đơn giản như sau:

- **Điều khiển người chơi:**

- Nhấn phím **W** để di chuyển người chơi lên trên.
- Nhấn phím **S** để di chuyển người chơi xuống dưới.

- **Bắn mũi tên:**

- Nhấn phím **Space (phím cách)** để bắn mũi tên từ vị trí của người chơi sang phải.

- **Mục tiêu trò chơi:**

- Người chơi cần bắn trúng các bóng bay đang di chuyển từ phải sang trái màn hình.

- **Điều kiện thắng:**

- Bắn trúng **2** quả bóng bay (biến 'hits' = 2). Đây là giá trị để dễ dàng kiểm thử, có thể tăng lên để trò chơi thử thách hơn.

- **Điều kiện thua:**

- Để lọt **5** quả bóng bay (biến ‘miss’ = 5). Tương tự như điều kiện thắng, giá trị này có thể được điều chỉnh.

- **Chơi lại hoặc thoát:**

- Sau khi thắng hoặc thua, màn hình sẽ hiển thị tùy chọn chơi lại.
- Nhấn phím **Y** hoặc **y** để chơi lại.
- Nhấn phím **N** hoặc **n** để thoát khỏi trò chơi.

- **Phím debug :**

- Trong một số phiên bản, phím mũi tên trái có thể được dùng để tăng số lần miss nhằm mục đích kiểm thử.

Chương 3

Kết Quả Thực Thi Và Hình Ảnh Minh Họa Trò Chơi

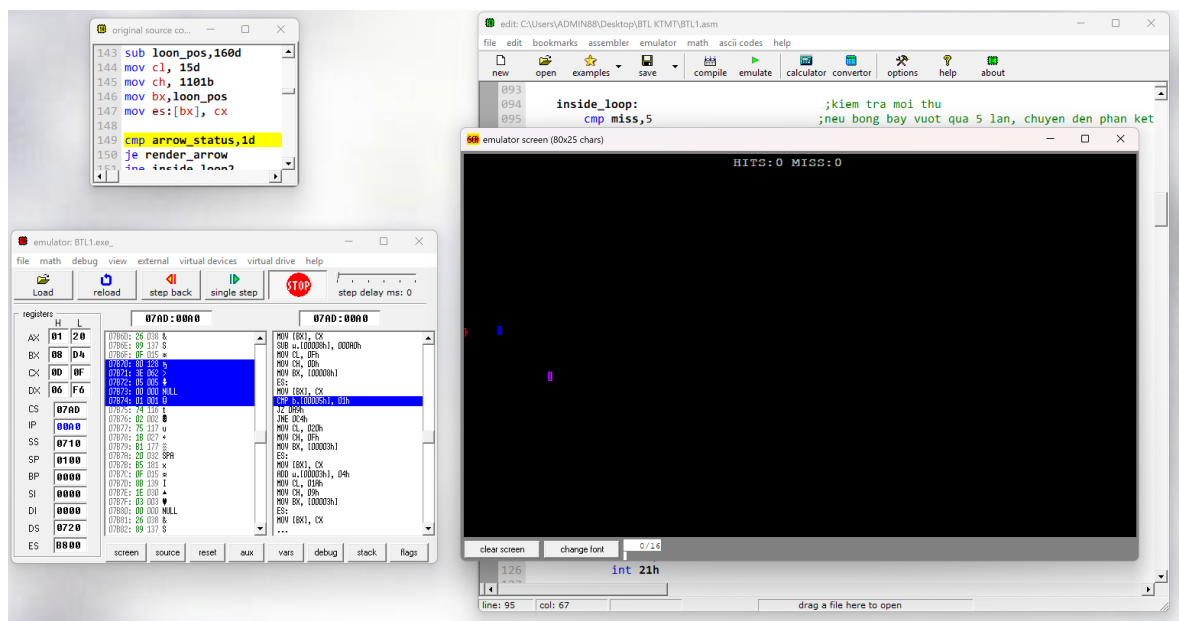
Phần này trình bày một số hình ảnh minh họa giao diện và các trạng thái của trò chơi khi thực thi trên trình giả lập Emu8086.



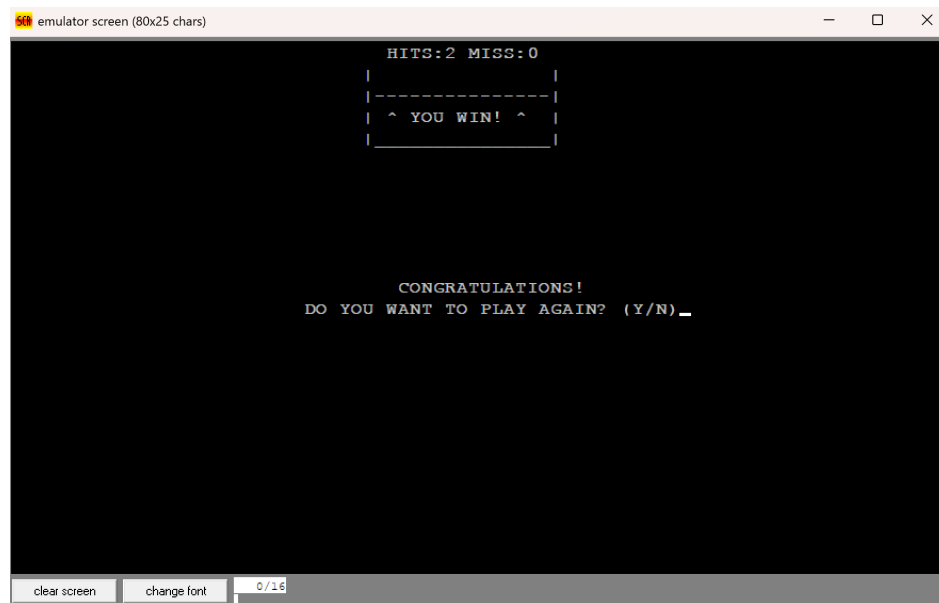
Hình 3.1: Giao diện menu chính của trò chơi.



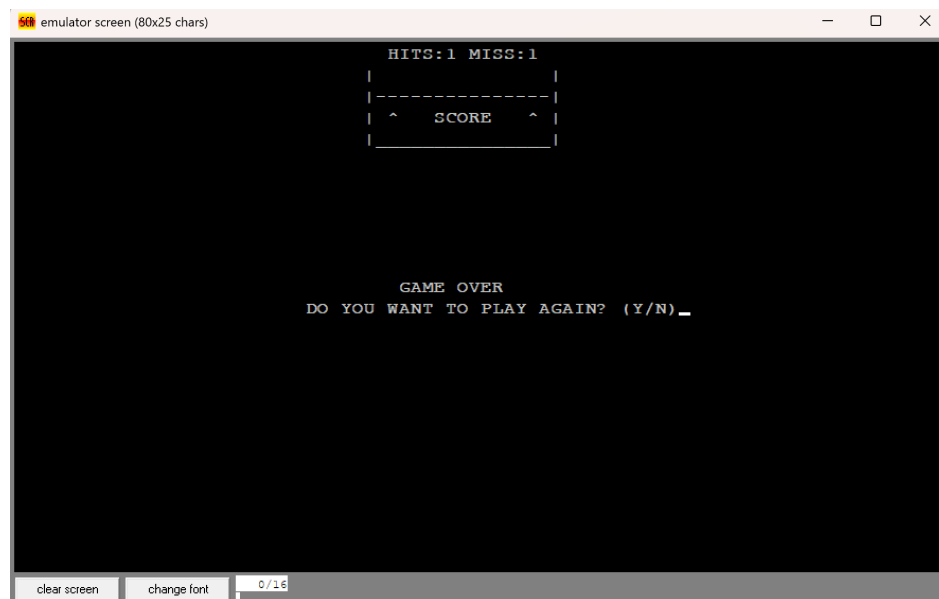
Hình 3.2: Giao diện trong trò chơi.



Hình 3.3: Toàn cảnh trò chơi đang diễn ra trong trình giả lập Emu8086, bao gồm màn hình game, mã nguồn và các thanh ghi.



Hình 3.4: Màn hình thông báo chiến thắng khi người chơi đạt đủ điểm.



Hình 3.5: Màn hình thông báo thua cuộc khi người chơi để lọt quá nhiều bóng bay.

Các hình ảnh trên cho thấy các giao diện cơ bản của trò chơi, từ lúc bắt đầu, trong quá trình chơi, cho đến khi kết thúc một lượt chơi với kết quả thắng hoặc thua. Giao diện được thiết kế đơn giản bằng các ký tự đồ họa trong chế độ text mode của Assembly.

Chương 4

Phân Tích Cấu Trúc Mã Nguồn Balloon Shooting Game

Chương trình Balloon Shooting Game được viết bằng ngôn ngữ Assembly cho vi xử lý 8086, sử dụng trình biên dịch EMU8086. Cấu trúc tổng quan của mã nguồn bao gồm các thành phần chính sau:

- **.model large:** Khai báo mô hình bộ nhớ lớn (large memory model). Mô hình này cho phép chương trình sử dụng nhiều đoạn mã (code segments) và nhiều đoạn dữ liệu (data segments) khác nhau, mỗi đoạn có thể lên tới 64KB. Điều này cung cấp không gian địa chỉ rộng hơn cho các chương trình phức tạp.
- **.stack 100h:** Dành riêng một vùng nhớ có kích thước 100h (256 bytes) cho ngăn xếp (stack). Ngăn xếp được sử dụng để lưu trữ địa chỉ trả về của các thủ tục, các biến cục bộ, và các tham số truyền cho thủ tục.
- **.data:** Khu vực khai báo các biến và hằng số mà chương trình sử dụng. Trong trò chơi này, phần .data chứa các biến lưu trữ vị trí của người chơi, mũi tên, bóng bay; trạng thái của các đối tượng; điểm số (số lần bắn trúng, số lần bắn trượt); và các chuỗi ký tự dùng để hiển thị thông báo trên màn hình (menu game, thông báo thắng/thua, điểm số).
- **.code:** Chứa toàn bộ mã lệnh thực thi của chương trình. Phần này bao gồm thủ tục chính (main proc) là điểm bắt đầu của chương trình, cùng với các thủ tục con và các nhãn (labels) khác để thực hiện các chức năng cụ

thể của trò chơi như hiển thị menu, xử lý sự kiện bàn phím, di chuyển đối tượng, kiểm tra va chạm, cập nhật điểm số, và hiển thị màn hình kết thúc game.

Sơ đồ luồng chương trình

- `main proc`
 - `game_menu` (Quản lý menu)
 - * Hiển thị menu.
 - * Chờ nhấn Enter.
 - * Gọi `clear_screen proc.`
 - * Gọi `show_score proc.`
 - `main_loop` (Vòng lặp chính)
 - * Kiểm tra phím nhấn.
 - `key_pressed`: Xử lý phím (W, S, Space, Left).
 - * `inside_loop` (Logic game cốt lõi)
 - Kiểm tra thua (`miss >= 5`).
 - Kiểm tra va chạm (`arrow_pos == loon_pos`).
 - Cập nhật vị trí: người chơi, mũi tên, bóng bay.
 - Vẽ đối tượng.
 - * `game_over` (Xử lý thua)
 - Hiển thị thông báo thua.
 - Hỏi chơi lại (Y/N).
 - * `win_game` (Xử lý thắng)
 - Hiển thị thông báo thắng.
 - Hỏi chơi lại (Y/N).
 - `show_score proc` (Tiện ích hiển thị điểm)
 - * Cập nhật chuỗi điểm số.

- `clear_screen` `proc` (Tiện ích xóa màn hình)

- * Xóa màn hình console.

Tổng quan, chương trình được tổ chức thành các module chức năng thông qua các thủ tục và nhãn, giúp mã nguồn dễ đọc và quản lý hơn.

Chương 5

Phân Tích Chi Tiết Chức Năng

Phần này mô tả chi tiết các thủ tục, nhãn và biến quan trọng trong mã nguồn.

5.1 Chi tiết các Thủ tục Chính (Procedures)

- **main proc:** Đây là thủ tục chính, điểm khởi đầu của chương trình. Nó thực hiện việc khởi tạo các đoạn dữ liệu (DS) và đoạn màn hình (ES), sau đó nhảy đến nhãn 'game_menu' để hiển thị menu bắt đầu trò chơi.
- **show_score proc:** Thủ tục này chịu trách nhiệm hiển thị điểm số hiện tại của người chơi, bao gồm số lần bắn trúng (Hits) và số lần bắn trượt (Miss). Điểm số được định dạng thành chuỗi "Hits:X Miss:Y" và lưu vào biến 'state_buf'.
- **clear_screen proc:** Thủ tục này dùng để xóa toàn bộ nội dung trên màn hình. Nó sử dụng ngắt 10h của BIOS với AH=0, AL=3 (chế độ văn bản 80x25 màu).

5.2 Giải thích các Biến (Variables) Dữ liệu Chính

- **hits dw 0d:** Lưu số lần người chơi bắn trúng bóng bay. Điều kiện thắng là khi 'hits' đạt 2 (trong phiên bản test, có thể tăng lên để khó hơn).
- **miss dw 0d:** Lưu số lần người chơi bắn trượt bóng bay (bóng bay bay qua màn hình). Điều kiện thua là khi 'miss' đạt 5 (trong phiên bản test, có thể

là 1 hoặc 7 tùy theo mô tả, mã nguồn ghi là 5).

- **player_pos dw 1760d**: Lưu vị trí (offset trong bộ nhớ video) của người chơi trên màn hình. Giá trị ban đầu là 1760d.
- **arrow_pos dw 0d**: Lưu vị trí của mũi tên. Giá trị 0d có nghĩa là mũi tên chưa được bắn hoặc đã biến mất.
- **loon_pos dw 3860d**: Lưu vị trí của bóng bay. Giá trị ban đầu là 3860d (bên phải màn hình).
- **arrow_status db 0d**: Trạng thái của mũi tên. 0d nghĩa là mũi tên sẵn sàng để bắn, 1d nghĩa là mũi tên đang bay.
- **arrow_limit dw 22d**: Giới hạn di chuyển của mũi tên. Khi 'arrow_pos' vượt qua 'player_pos + arrow_limit', mũi tên sẽ biến mất. Giá trị 22d là một khoảng cách ngắn, có thể điều chỉnh.
- **direction db 0d**: Lưu hướng di chuyển mà người chơi yêu cầu (8d cho lên, 2d cho xuống, 0d là không di chuyển).
- **state_buf db '00:0:0:0:0:0:00:00\$'**: Bộ đệm để lưu chuỗi điểm số hiển thị.
- **game_over_str, game_start_str, win_str**: Các chuỗi ký tự dùng để hiển thị thông báo tương ứng.

5.3 Phân tích các Nhãn (Labels) Quan trọng

- **game_menu**: Hiển thị giao diện menu chính của trò chơi, bao gồm tên game, thông tin nhóm phát triển, hướng dẫn cách chơi cơ bản. Chờ người chơi nhấn phím Enter để bắt đầu.

```

1 game_menu:
2     mov ah, 09h
3     mov dh, 0
4     mov dx, offset game_start_str ;tro den chuoi bat dau tro choi
5     int 21h                       ;goi ngat 21h de hien thi chuoi
6 ;nhan dau vao tu nguoi choi
7     input2:

```

```

8      mov ah,1
9      int 21h
10     cmp al,13d      ;so sanh ky tu nhap voi ma phim Enter
11     jne input2      ;neu khong phai Enter thi cho nhap lai
12     call clear_screen ;goi thu tuc xoa man hinh
13
14     lea bx,state_buf ;gan dia chi diem so vao bx
15     call show_score  ;goi thu tuc hien thi diem so
16     lea dx,state_buf ;gan dia chi diem vao Dx de hien thi
17     mov ah,09h
18     int 21h          ;goi ngat 21h de hien thi diem
19
20     mov ah,2          ;ham 02h-xuat mot ky tu ra man hinh
21     mov dl, 0dh
22     int 21h
23     jmp main_loop     ;nhay den vong lap chinh tro choi

```

Listing 5.1: Mã nguồn nhân game_menu

- **main_loop:** Vòng lặp chính của trò chơi. Trong mỗi vòng lặp, chương trình kiểm tra xem có phím nào được nhấn không (qua ngắt 16h). Nếu có, nhảy đến ‘key_pressed’ để xử lý. Nếu không, tiếp tục thực hiện logic trong ‘inside_loop’.

```

1 main_loop:      ;Cap nhat logic va hien thi moi thu
2     mov ah,1h
3     int 16h      ;Kiem tra neu co phim duoc nhan
4     jnz key_pressed ;Neu co,nhay den key_pressed
5     jmp inside_loop ;Neu khong,nhay den inside_loop

```

Listing 5.2: Mã nguồn nhân game_loop

- **inside_loop:** Thực hiện các công việc cốt lõi của game trong mỗi frame: kiểm tra điều kiện thua (miss $\geq$ 5), kiểm tra va chạm giữa mũi tên và bóng bay, cập nhật vị trí người chơi dựa trên biến ‘direction’, ấn mũi tên nếu vượt quá giới hạn, kiểm tra bóng bay có bay qua màn hình không. Sau đó gọi các nhân để vẽ lại đối tượng.

```

1 inside_loop:      ;kiem tra moi thu
2     cmp miss,5      ; dieu kien thua,thay doi tuy y
3     jge game_over
4
5     mov dx,arrow_pos ;kiem tra va cham
6     cmp dx, loon_pos
7     je hit
8
9     cmp direction,8d ;cap nhat vi tri nguoi choi
10    je player_up
11    cmp direction,2d ;len hoac xuong dua tren bien huong

```

```

12     je player_down
13
14     mov dx,arrow_limit      ;an mui ten
15     cmp arrow_pos, dx
16     jge hide_arrow
17
18     cmp loon_pos, 0d        ;kiem tra bong bay vuot qua
19     jle miss_loon
20     jne render_loon

```

Listing 5.3: Mã nguồn nhãn `inside_loop`

- **hit:** Được gọi khi mũi tên bắn trúng bóng bay. Thủ tục này phát âm thanh nhỏ, tăng biến 'hits', cập nhật và hiển thị lại điểm số. Kiểm tra nếu 'hits' đạt 2 thì chuyển đến màn hình thắng ('win_game'). Sau đó, gọi 'fire_loon' để tạo bóng bay mới.

```

1 hit:
2     mov ah,2                ;in ra ky tu } tren man hinh
3     mov dx, 7d
4     int 21h
5
6     inc hits                ;cap nhat diem so
7
8     lea bx,state_buf        ;hien thi diem so ngay lap tuc
9     call show_score
10    lea dx,state_buf
11    mov ah,09h
12    int 21h
13
14    mov ah,2                ;xuong dong
15    mov dl, 0dh
16    int 21h
17
18    cmp hits, 2              ;dieu kien thang,thay doi tuy y
19    je win_game
20
21    jmp fire_loon           ;bong bay moi xuat hien

```

Listing 5.4: Mã nguồn nhãn `hit`

- **render_loon:** Di chuyển bóng bay từ phải sang trái và vẽ lại bóng bay ở vị trí mới. Trước đó, xóa hình ảnh bóng bay ở vị trí cũ.

```

1 render_loon:                ;ve bong bay
2     mov cl, ' '             ;an bong bay cu
3     mov ch, 1111b
4     mov bx,loon_pos
5     mov es:[bx], cx
6
7     sub loon_pos,160d        ;va ve bong bay moi o vi tri moi

```

```

8      mov cl, 15d
9      mov ch, 1101b
10     mov bx, loon_pos
11     mov es:[bx], cx
12
13     cmp arrow_status, 1d ;kiem tra xem co mui ten nao de ve khong
14     je render_arrow
15     jne inside_loop2

```

Listing 5.5: Mã nguồn nhãn render_loon

- **render_arrow:** Nếu mũi tên đang được bắn ('arrow_status = 1'), thủ tục này di chuyển mũi tên sang phải và vẽ lại mũi tên ở vị trí mới. Trước đó, xóa hình ảnh mũi tên ở vị trí cũ.

```

1 render_arrow:                ;ve mui ten
2     mov cl, ' '
3     mov ch, 1111b
4     mov bx, arrow_pos        ;an vi tri cu
5     mov es:[bx], cx
6
7     add arrow_pos, 4d        ;ve vi tri moi
8     mov cl, 26d
9     mov ch, 1001b
10    mov bx, arrow_pos
11    mov es:[bx], cx

```

Listing 5.6: Mã nguồn nhãn render_arrow

- **player_up:** Di chuyển người chơi lên trên một khoảng cách nhất định bằng cách trừ giá trị 'player_pos' đi 160 (một dòng trên màn hình text mode). Đặt lại 'direction' về 0.

```

1 player_up:
2     mov cl, ' '                ;an vi tri cu cua nguoi choi
3     mov ch, 1111b
4     mov bx, player_pos
5     mov es:[bx], cx
6
7     sub player_pos, 160d        ;dat vi tri moi cho nguoi choi
8     mov direction, 0
9     jmp inside_loop2           ;no se duoc ve trong vong lap chinh

```

Listing 5.7: Mã nguồn nhãn player_up

- **player_down:** Di chuyển người chơi xuống dưới một khoảng cách nhất định bằng cách cộng giá trị 'player_pos' thêm 160. Đặt lại 'direction' về 0.

```

1 player_down:
2     mov cl, ' '           ;giống như player_up
3     mov ch, 1111b         ;an cái cu va dat vi tri moi
4     mov bx,player_pos
5     mov es:[bx], cx
6
7     add player_pos,160d    ;va vong lap chinh se ve no
8     mov direction, 0
9     jmp inside_loop2

```

Listing 5.8: Mã nguồn nhân player_down

- **key_pressed:** Xử lý các phím người chơi nhấn. Kiểm tra mã phím (AH) để xác định hành động: phím 'W' (11h) gọi 'upKey', phím 'S' (1Fh) gọi 'downKey', phím Space (39h) gọi 'spaceKey' (bắn tên), phím mũi tên trái (4Bh) dùng để debug (tăng 'miss').

```

1 key_pressed:           ;phan xu ly dau vao
2     mov ah,0
3     int 16h
4
5     cmp ah,11h ;11h la Scan Code (hex) cua W
6     je upKey     ;chuyen den upKey neu duoc nhan
7     cmp ah,1Fh ;1Fh la Scan Code (hex) cua S
8     je downKey   ;chuyen den downKey neu duoc nhan
9
10    cmp ah,39h    ;chuyen den spaceKey neu nut space duoc nhan
11    je spaceKey
12
13    cmp ah,4Bh    ;chuyen den leftKey (dung de debug)
14    je leftKey
15    jmp inside_loop ;neu khong nhan nut nao,nhay den inside_loop

```

Listing 5.9: Mã nguồn nhân key_pressed

- **upKey:** Đặt biến 'direction' thành 8 (hướng lên).

```

1 upKey:           ;dat huong di chuyen cua nguoi choi len tren
2     mov direction, 8d
3     jmp inside_loop

```

Listing 5.10: Mã nguồn nhân upKey

- **downKey:** Đặt biến 'direction' thành 2 (hướng xuống).

```

1 downKey:           ;dat huong di chuyen cua nguoi choi xuong duoi
2     mov direction, 2d
3     jmp inside_loop

```

Listing 5.11: Mã nguồn nhân downKey

- **spaceKey / fire_arrow**: Nếu mũi tên chưa được bắn ('arrow_status = 0'), thủ tục này sẽ bắn mũi tên. Nó đặt vị trí ban đầu của mũi tên ('arrow_pos') bằng vị trí hiện tại của người chơi ('player_pos'), đặt giới hạn di chuyển của mũi tên ('arrow_limit'), và cập nhật 'arrow_status' thành 1 (đang bắn).

```

1 spaceKey:                ;ban mui ten
2     cmp arrow_status,0
3     je  fire_arrow
4     jmp inside_loop
5
6 fire_arrow:              ;dat vi tri mui ten tai vi tri nguoi
   Choi
7     mov dx, player_pos   ;de mui ten ban tu vi tri nguoi Choi
8     mov arrow_pos, dx
9
10    mov dx,player_pos    ;khi ban mui ten, no cung dat gioi han
11    mov arrow_limit, dx  ;cua mui ten, noi ma no se bi an
12    add arrow_limit, 22d ;150
13
14    mov arrow_status, 1d  ;dat trang thai mui ten,ngan chan ban
   nhieu lan
15    jmp inside_loop

```

Listing 5.12: Mã nguồn nhân spaceKey/fire_arrow

- **miss_loon**: Được gọi khi bóng bay bay qua hết màn hình mà không bị bắn trúng. Tăng biến 'miss', cập nhật và hiển thị điểm số. Sau đó gọi 'fire_loon' để tạo bóng bay mới.

```

1 miss_loon:
2     add miss,1           ;cap nhat diem so
3     lea bx,state_buf    ;hien thi diem so
4     call show_score
5     lea dx,state_buf
6     mov ah,09h
7     int 21h
8
9     ;xuong dong
10    mov ah,2
11    mov dl, 0dh
12    int 21h
13    jmp fire_loon

```

Listing 5.13: Mã nguồn nhân miss_loon

- **fire_loon**: Tạo một bóng bay mới bằng cách đặt lại vị trí 'loon_pos' về giá trị ban đầu (3860d, ở biên phải màn hình) và đặt 'loon_status' thành 1.

```

1 fire_loon:                ;ban bong bay moi
2     mov loon_status, 1d
3     mov loon_pos, 3860d    ;3990d
4     jmp render_loon

```

Listing 5.14: Mã nguồn nhãn fire_loon

- **hide_arrow:** Khi mũi tên bay đến giới hạn ('arrow_limit'), thủ tục này sẽ ẩn mũi tên bằng cách vẽ ký tự rỗng tại vị trí đó và đặt 'arrow_status' về 0 (sẵn sàng bắn).

```

1 hide_arrow:
2     mov arrow_status, 0    ;an mui ten
3     mov cl, ' '
4     mov ch, 1111b
5     mov bx, arrow_pos
6     mov es:[bx], cx
7
8     cmp loon_pos, 0d
9     jle miss_loon
10    jne render_loon
11    jmp inside_loop2

```

Listing 5.15: Mã nguồn nhãn hide_arrow

- **game_over:** Hiện thị màn hình thông báo thua cuộc. Xóa các đối tượng game khỏi màn hình. Reset lại các biến game về giá trị ban đầu. Hỏi người chơi có muốn chơi lại không (Y/N). Nếu chọn 'Y' hoặc 'y', gọi 'play_again'. Nếu chọn 'N' hoặc 'n', gọi 'exit_game'.

```

1 game_over:
2     mov cl, ' '           ; an bong bay
3     mov ch, 1111b
4     mov bx, loon_pos
5     mov es:[bx], cx
6
7     mov cl, ' '           ; an mui ten
8     mov ch, 1111b
9     mov bx, arrow_pos
10    mov es:[bx], cx
11
12    mov cl, ' '           ; an nguoi choi
13    mov ch, 1111b
14    mov bx, player_pos
15    mov es:[bx], cx
16
17    mov ah, 09h           ; in chu "Game Over"
18    mov dx, offset game_over_str
19    int 21h
20

```

```

21     ; reset bien
22     mov miss, 0d
23     mov hits, 0d
24     mov player_pos, 1760d
25     mov arrow_pos, 0d
26     mov arrow_status, 0d
27     mov arrow_limit, 22d
28     mov loon_pos, 3860d
29     mov loon_status, 0d
30     mov direction, 0d

```

Listing 5.16: Mã nguồn nhả game_over

- **win_game**: Hiển thị màn hình thông báo thắng cuộc. Xóa các đối tượng game. Reset biến. Hỏi người chơi có muốn chơi lại không (Y/N).

```

1 win_game:
2     mov ah, 09h           ; in chu "You Win"
3     mov dx, offset win_str
4     int 21h
5
6     ; an mui ten
7     mov cl, ' '
8     mov ch, 1111b
9     mov bx, arrow_pos
10    mov es:[bx], cx
11
12    ; an nguoi choi
13    mov cl, ' '
14    mov ch, 1111b
15    mov bx, player_pos
16    mov es:[bx], cx
17
18    ; reset bien
19    mov miss, 0d
20    mov hits, 0d
21    mov player_pos, 1760d
22    mov arrow_pos, 0d
23    mov arrow_status, 0d
24    mov arrow_limit, 22d
25    mov loon_pos, 3860d
26    mov loon_status, 0d
27    mov direction, 0d

```

Listing 5.17: Mã nguồn nhả win_game

- **play_again**: Xóa màn hình và nhảy về ‘main_loop’ để bắt đầu lượt chơi mới.

```

1 play_again:           ; choi lai
2     call clear_screen ; xoa man hinh
3     jmp main_loop     ; quay lai vong lap chinh

```

Listing 5.18: Mã nguồn nhả play_again

- **exit_game**: Kết thúc chương trình bằng ngắt 21h với AH=4Ch.

```
1 exit_game:                ; thoát game
2     mov ah,4Ch             ; ham thoát DOS
3     int 21h
```

Listing 5.19: Mã nguồn nhân exit_game

Chương 6

Tổng Kết, Đánh Giá Và Hướng Phát Triển

Qua quá trình thực hiện bài tập lớn "Game Bắn Bóng Bay (Balloon Shooting Game) bằng Assembly", nhóm đã đạt được những kết quả nhất định. Trước hết, nhóm đã tìm hiểu và nắm vững hơn các kiến thức cơ bản về lập trình Assembly cho vi xử lý 8086, bao gồm cấu trúc chương trình, cách sử dụng các thanh ghi, tập lệnh, quản lý bộ nhớ, và các kỹ thuật xử lý vào ra cơ bản.

Mã nguồn của trò chơi đã được xây dựng và chỉnh sửa dựa trên một phiên bản open-source, đảm bảo các chức năng cốt lõi như di chuyển người chơi, bắn mũi tên, tạo và di chuyển bóng bay, kiểm tra va chạm, tính điểm, và xử lý thắng/thua. Trò chơi hoạt động ổn định trên trình giả lập Emu8086.

Tuy nhiên, do giới hạn về thời gian và kiến thức, trò chơi vẫn còn ở mức độ đơn giản và có nhiều tiềm năng để phát triển thêm:

- **Nâng cao độ khó:** Tăng tốc độ bóng bay, thay đổi quỹ đạo di chuyển của bóng, hoặc yêu cầu số điểm cao hơn để thắng.
- **Thêm hiệu ứng:** Cải thiện hiệu ứng hình ảnh khi bắn trúng hoặc bắn trượt, thêm âm thanh đa dạng hơn.
- **Nhiều cấp độ (Levels):** Phát triển thêm các cấp độ với độ khó tăng dần.
- **Lưu điểm cao (High Score):** Thêm chức năng lưu lại điểm số cao nhất của người chơi.

- **Đa dạng đối tượng:** Thêm các loại bóng bay khác nhau với điểm số hoặc hành vi khác nhau, hoặc các vật phẩm hỗ trợ.

Nhìn chung, việc hoàn thành đề tài này đã mang lại nhiều kinh nghiệm quý báu trong việc lập trình ở mức độ thấp và hiểu rõ hơn về cách máy tính hoạt động. Đây là một nền tảng tốt để tiếp tục nghiên cứu các lĩnh vực liên quan đến hệ thống máy tính và phát triển phần mềm.

Phụ lục A

Phụ lục mã nguồn

Toàn bộ mã nguồn của trò chơi Balloon Shooting Game được trình bày dưới đây.

```
1 .model large
2 .stack 100h      ; Du tru 256 byte cho stack
3 .data
4
5 exit db 0
6 player_pos dw 1760d          ;vi tri cua nguoi choi
7
8 arrow_pos dw 0d              ;vi tri cua mui ten
9 arrow_status db 0d           ;0 = mui ten san sang
    ban, khac 0 thi khong
10 arrow_limit dw 22d          ;150d
11
12 loon_pos dw 3860d            ;3990d
13 loon_status db 0d
14
15                                ;huong di chuyen cua
    nguoi choi
16                                ;len=8, xuong=2
17 direction db 0d
18
19 state_buf db '00:0:0:0:0:0:00:00$' ;bien diem so
20 ;hit_num db 0d
21 hits dw 0d
22 miss dw 0d
23
24 game_over_str dw ' ',0ah,0dh
25 dw '                                | ',0ah,0dh
26 dw '                                | ',0ah,0dh
27 dw '                                | ^   Score   ^ | ',0ah,0dh
28 dw '                                | ',0ah,0dh
29 dw ' ',0ah,0dh
30 dw ' ',0ah,0dh
31 dw ' ',0ah,0dh
32 dw ' ',0ah,0dh
33 dw ' ',0ah,0dh
```

```

34 dw ' ',0ah,0dh
35 dw '                                     Game Over',0ah,0dh
36 dw '                                     Do you want to play again? (Y/N)$',0ah,0dh
37
38 game_start_str dw ' ',0ah,0dh
39 dw ' ',0ah,0dh
40 dw ' ',0ah,0dh
41 dw ' ',0ah,0dh
42 dw ' =====',0ah,0dh
43 dw ' ||',0ah,0dh
44 dw ' ||          *      Balloon Shooting Game      *',0ah,0dh
45 dw ' ||',0ah,0dh
46 dw ' ||-----',0ah,0dh
47 dw ' ||',0ah,0dh
48 dw ' ||          Game Team2',0ah,0dh
49 dw ' ||          This game was modified by Group BTL2',0ah,0dh
50 dw ' || (D23 PTIT Hanoi),based on an open-source version',0ah,0dh
51 dw ' ||',0ah,0dh
52 dw ' ||          Use W and S key to move player',0ah,0dh
53 dw ' ||          and space button to shoot',0ah,0dh
54 dw ' ||',0ah,0dh
55 dw ' ||          Miss 5 balloons and you lose',0ah,0dh
56 dw ' ||          Hit 2 balloons and you win',0ah,0dh
57 dw ' ||',0ah,0dh
58 dw ' ||          Press Enter to start',0ah,0dh
59 dw ' ||',0ah,0dh
60 dw ' ||',0ah,0dh
61 dw ' =====',0ah,0dh
62 dw '$',0ah,0dh
63
64 win_str dw ' ',0ah,0dh
65 dw ' |',0ah,0dh
66 dw ' |-----',0ah,0dh
67 dw ' | ^ You Win! ^',0ah,0dh
68 dw ' |-----',0ah,0dh
69 dw ' ',0ah,0dh
70 dw ' ',0ah,0dh
71 dw ' ',0ah,0dh
72 dw ' ',0ah,0dh
73 dw ' ',0ah,0dh
74 dw ' ',0ah,0dh
75 dw '                                     Congratulations!',0ah,0dh
76 dw '                                     Do you want to play again? (Y/N)$',0ah,0dh
77
78 .code
79 main proc
80 mov ax,@data
81 mov ds,ax
82
83 mov ax, 0B800h
84 mov es,ax
85
86 jmp game_menu ;hien thi menu chinh
87

```

```

88 main_loop:                                ;cap nhat logic va hien
    thi moi thu
89     mov ah,1h
90     int 16h                                ;kiem tra neu co phim duoc
    nhan
91     jnz key_pressed
92     jmp inside_loop                        ;hoac tiep tuc
93
    inside_loop:
94         ;kiem tra moi thu
95         cmp miss,5                        ;neu bong bay vuot qua 5
    lan, chuyen den phan ket thuc tro choi
96         jge game_over
97
98         mov dx,arrow_pos                  ;kiem tra va cham
99         cmp dx, loon_pos
100        je hit
101
102        cmp direction,8d                  ;cap nhat vi tri nguoi choi
103        je player_up
104        cmp direction,2d                  ;len hoac xuong dua tren
    bien huong
105        je player_down
106
107        mov dx,arrow_limit                ;an mui ten
108        cmp arrow_pos, dx
109        jge hide_arrow
110
111        cmp loon_pos, 0d                  ;kiem tra bong bay vuot qua
112        jle miss_loon
113        jne render_loon
114
115        hit:                              ;phat am thanh neu ban
    trung
116            mov ah,2
117            mov dx, 7d
118            int 21h
119
120            inc hits                      ;cap nhat diem so
121
122            lea bx,state_buf              ;hien thi diem so ngay lap
    tuc
123            call show_score
124            lea dx,state_buf
125            mov ah,09h
126            int 21h
127
128            mov ah,2                      ;xuong dong
129            mov dl, 0dh
130            int 21h
131
132            cmp hits, 2                  ;kiem tra neu so lan ban
    trung dat 2
133            je win_game
134
135            jmp fire_loon                ;bong bay moi xuat hien

```

```

136
137         render_loon:                                ;ve bong bay
138             mov cl, ' '                               ;an bong bay cu
139             mov ch, 1111b
140             mov bx,loon_pos
141             mov es:[bx], cx
142
143             sub loon_pos,160d                           ;va ve bong bay moi o vi
tri moi
144             mov cl, 15d
145             mov ch, 1101b
146             mov bx,loon_pos
147             mov es:[bx], cx
148
149             cmp arrow_status,1d                         ;kiem tra xem co mui ten
nao de ve khong
150             je render_arrow
151             jne inside_loop2
152
153         render_arrow:                                ;ve mui ten
154             mov cl, ' '
155             mov ch, 1111b
156             mov bx,arrow_pos                           ;an vi tri cu
157             mov es:[bx], cx
158
159             add arrow_pos,4d                             ;ve vi tri moi
160             mov cl, 26d
161             mov ch, 1001b
162             mov bx,arrow_pos
163             mov es:[bx], cx
164
165         inside_loop2:
166             mov cl, 125d                               ;ve nguoi choi
167             mov ch, 1100b
168             mov bx,player_pos
169             mov es:[bx], cx
170
171             cmp exit,0
172             je main_loop                               ;ket thuc vong lap chinh
173             jmp program_end
174
175     jmp inside_loop2
176
177 player_up:                                           ;an vi tri cu cua nguoi choi
178     mov cl, ' '
179     mov ch, 1111b
180     mov bx,player_pos
181     mov es:[bx], cx
182
183     sub player_pos, 160d                             ;dat vi tri moi cho nguoi
choi
184     mov direction, 0
185     jmp inside_loop2                               ;no se duoc ve trong vong
lap chinh
186

```

```

187 player_down:
188     mov cl, ' ' ;giống như player_up
189     mov ch, 1111b ;an cai cu va dat vi tri moi
190     mov bx,player_pos
191     mov es:[bx], cx
192
193     add player_pos,160d ;va vong lap chinh se ve no
194     mov direction, 0
195     jmp inside_loop2
196
197 key_pressed: ;phan xu ly dau vao
198     mov ah,0
199     int 16h
200
201     cmp ah,11h ;chuyen den upKey neu nut
202     len duoc nhan
203     je upKey
204     cmp ah,1Fh
205     je downKey
206
207     cmp ah,39h ;chuyen den spaceKey neu
208     nut space duoc nhan
209     je spaceKey
210
211     cmp ah,4Bh ;chuyen den leftKey (dung
212     de debug)
213     je leftKey
214     jmp inside_loop ;neu khong co nut nao duoc
215     nhan, chuyen den ben trong vong lap
216
217 leftKey: ;chung ta dung no de debug
218     inc miss
219     lea bx,state_buf
220     call show_score
221     lea dx,state_buf
222     mov ah,09h
223     int 21h
224
225     mov ah,2
226     mov dl, 0dh
227     int 21h
228     jmp inside_loop
229
230 upKey: ;dat huong di chuyen cua
231     nguoi choi len tren
232     mov direction, 8d
233     jmp inside_loop
234
235 downKey:
236     mov direction, 2d ;dat huong di chuyen cua
237     nguoi choi xuong duoi
238     jmp inside_loop
239
240 spaceKey: ;ban mui ten
241     cmp arrow_status,0

```

```

236     je    fire_arrow
237     jmp   inside_loop
238
239 fire_arrow:                                ;dat vi tri mui ten tai vi
    tri nguoi choi
240     mov dx, player_pos                    ;de mui ten ban tu vi tri
    nguoi choi
241     mov arrow_pos, dx
242
243     mov dx, player_pos                    ;khi ban mui ten, no cung
    dat gioi han
244     mov arrow_limit, dx                  ;cua mui ten, noi ma no se
    bi an
245     add arrow_limit, 22d    ;150
246
247     mov arrow_status, 1d                  ;dat trang thai mui ten. No
    ngan chan viec ban nhieu lan
248     jmp   inside_loop                    ;ban
249
250 miss_loon:
251     add miss, 1                           ;cap nhat diem so
252     lea bx, state_buf                     ;hien thi diem so
253     call show_score
254     lea dx, state_buf
255     mov ah, 09h
256     int 21h
257
258     mov ah, 2
259     mov dl, 0dh
260     int 21h
261     jmp   fire_loon
262
263 fire_loon:                                ;ban bong bay moi
264     mov loon_status, 1d
265     mov loon_pos, 3860d    ;3990d
266     jmp   render_loon
267
268 hide_arrow:
269     mov arrow_status, 0                    ;an mui ten
270     mov cl, ' '
271     mov ch, 1111b
272     mov bx, arrow_pos
273     mov es:[bx], cx
274
275     cmp loon_pos, 0d
276     jle miss_loon
277     jne render_loon
278     jmp   inside_loop2
279
280 game_over:
281     mov cl, ' '                           ;an bong bay cuoi cung tren
    man hinh
282     mov ch, 1111b
283     mov bx, loon_pos
284     mov es:[bx], cx

```

```

285
286     mov cl, ' '                                ;an mui ten tren man hinh
287     mov ch, 1111b
288     mov bx,arrow_pos
289     mov es:[bx], cx
290
291     mov cl, ' '                                ;an nguoi choi
292     mov ch, 1111b
293     mov bx,player_pos
294     mov es:[bx], cx
295
296     mov ah,09h
297     mov dx, offset game_over_str
298     int 21h
299
300     ;cap nhat bien de bat dau lai
301     mov miss, 0d
302     mov hits,0d
303     mov player_pos, 1760d
304     mov arrow_pos, 0d
305     mov arrow_status, 0d
306     mov arrow_limit, 22d                        ;150d
307     mov loon_pos, 3860d                          ;3990d
308     mov loon_status, 0d
309     mov direction, 0d
310
311     input:
312         mov ah,1
313         int 21h
314         cmp al,'Y'
315         je play_again
316         cmp al,'y'
317         je play_again
318         cmp al,'N'
319         je exit_game
320         cmp al,'n'
321         je exit_game
322         jmp input                                ;neu khong phai Y/N, cho
323     dau vao hop le
324
325     play_again:
326         call clear_screen
327         jmp main_loop
328
329     exit_game:
330         mov ah,4Ch
331         int 21h
332 win_game:
333     mov ah,09h
334     mov dx, offset win_str
335     int 21h
336
337     ; An mui ten va nguoi choi
338     mov cl, ' '

```

```
339     mov ch, 1111b
340     mov bx, arrow_pos
341     mov es:[bx], cx
342
343     mov cl, ' '
344     mov ch, 1111b
345     mov bx, player_pos
346     mov es:[bx], cx
347
348     ; Dat lai cac bien
349     mov miss, 0d
350     mov hits, 0d
351     mov player_pos, 1760d
352     mov arrow_pos, 0d
353     mov arrow_status, 0d
354     mov arrow_limit, 22d
355     mov loon_pos, 3860d
356     mov loon_status, 0d
357     mov direction, 0d
358
359     input_win:
360         mov ah,1
361         int 21h
362         cmp al,'Y'
363         je play_again
364         cmp al,'y'
365         je play_again
366         cmp al,'N'
367         je exit_game
368         cmp al,'n'
369         je exit_game
370         jmp input_win
371
372     game_menu:
373         mov ah,09h
374         mov dh,0
375         mov dx, offset game_start_str
376         int 21h
377
378                                     ;cho dau vao
379     input2:
380         mov ah,1
381         int 21h
382         cmp al,13d
383         jne input2
384         call clear_screen
385
386                                     ;hien thi diem so
387         lea bx,state_buf
388         call show_score
389         lea dx,state_buf
390         mov ah,09h
391         int 21h
392
393         mov ah,2
394         mov dl, 0dh
395         int 21h
```

```
394         jmp main_loop
395
396 program_end:                                ;ket thuc tro choi:)
397 mov exit,10d
398
399 main endp
400
401 show_score proc
402     lea bx,state_buf
403     mov dx,hits
404     add dx,48d
405     mov [bx],9d
406     mov [bx+1],9d
407     mov [bx+2],9d
408     mov [bx+3],9d
409     mov [bx+4],'H'
410     mov [bx+5],'i'
411     mov [bx+6],'t'
412     mov [bx+7],'s'
413     mov [bx+8],':'
414     mov [bx+9],dx
415     mov dx,miss
416     add dx,48d
417     mov [bx+10],''
418     mov [bx+11],'M'
419     mov [bx+12],'i'
420     mov [bx+13],'s'
421     mov [bx+14],'s'
422     mov [bx+15],':'
423     mov [bx+16],dx
424 ret
425 show_score endp
426
427 clear_screen proc near
428     mov ah,0
429     mov al,3
430     int 10h
431     ret
432 clear_screen endp
433
434 end main
```

Listing A.1: Mã nguồn Game

Tài liệu tham khảo

- [1] Rezve. *8086 Microprocessor Game in Assembly Language*. GitHub repository. <https://github.com/Rezve/8086-Microprocessor-Game-in-Assembly-Language>
- [2] Khoa Công nghệ Thông tin. *Bài giảng Kiến trúc Máy tính*. Học viện Công nghệ Bưu chính Viễn thông, 2023.